

1. Contexte et objectifs

Pépi+ est une société de pépinière dont la gestion des stocks, des commandes et des partenaires fournisseurs était entièrement manuelle (tableurs, papier). Cette absence d'outil dédié rendait impossible la traçabilité sanitaire réglementaire des plants : chaque plant vendu doit pouvoir être retracé par sa pépinière de provenance et sa saison de production. Ce projet consiste à développer une application web complète répondant à ce besoin métier, destinée à un personnel sans compétence informatique — l'ergonomie et la simplicité d'utilisation sont donc des contraintes aussi importantes que les fonctionnalités techniques.

Deux types de stocks coexistent dans un seul outil : le stock réel (production interne Pépi+) et le stock virtuel (partenaires fournisseurs). Le collaborateur crée une commande en piochant indifféremment dans les deux sources. Les quantités réservées sont décrémentées immédiatement à la création de la commande, sans aucune transformation supplémentaire lors de la livraison physique — la livraison ne fait que confirmer le changement de statut.

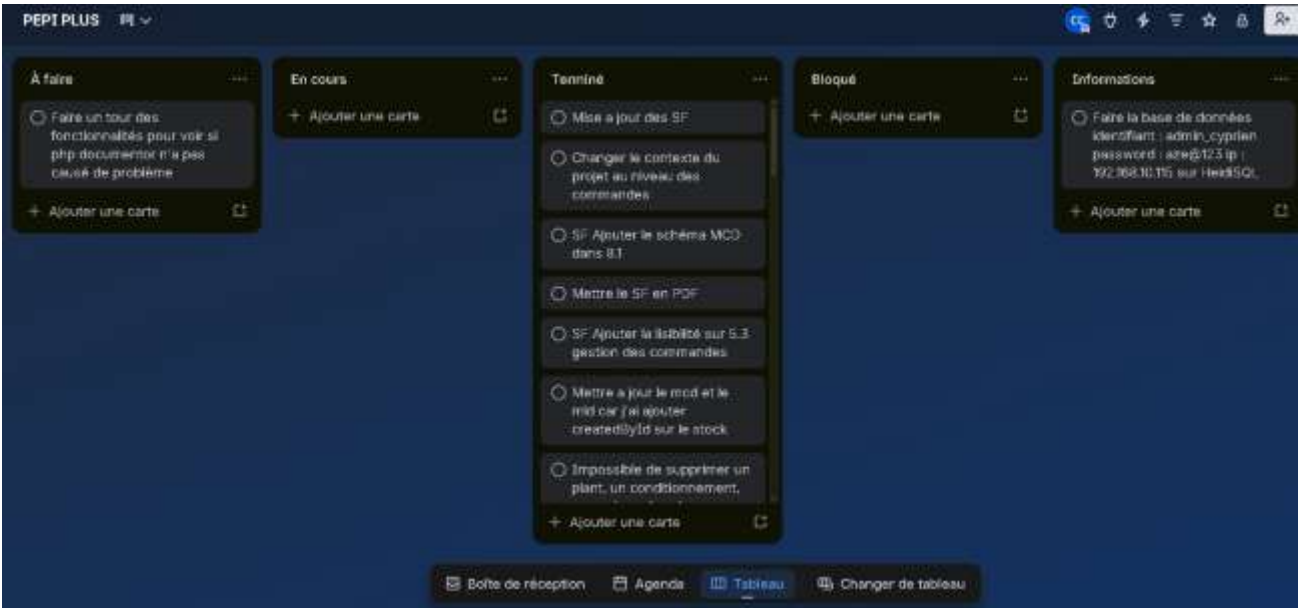
Périmètre du projet

INCLUS	Application web interne (collaborateurs, admin) + espace partenaire fournisseur — Gestion stock réel (Pépi+) et virtuel (partenaires) dans une vue unifiée — Traçabilité sanitaire obligatoire : provenance + saison — Cycle complet des commandes : panier → réservation → livraison ou annulation — Tableau de bord avec alertes stocks critiques — Gestion du référentiel : plants, conditionnements, saisons — Gestion des utilisateurs, rôles et partenaires
EXCLU	Application mobile native (iOS / Android) — Module de facturation ou de paiement — Intégration avec un ERP ou logiciel comptable externe — Gestion des livraisons physiques — Vente directe au public (e-commerce)

2. Architecture technique

Couche	Technologie	Version
Backend	Symfony	7.4 (PHP 8.2+)
ORM / BDD	Doctrine ORM + MySQL	3.6 + MySQL 8.0
Migrations BDD	Doctrine Migrations	3.7
Templating	Twig	3.x
CSS / UI	TailwindCSS	4.1
Build assets	Webpack Encore	2.4
Pagination	KnplPaginatorBundle	6.10
Emails	Symfony Mailer + Mailtrap	7.4
Reset password	symfonycasts/reset-password-bundle	1.24
Documentation	PHPDocumentor	3.9.1
Versionning	Git / GitHub	—
Gestion de projet	Trello	—

TRELLO :



Structure des sources

Dossier	Contenu
src/Command/	Commandes console personnalisées (SetupDatabaseCommand)
src/Controller/	12 contrôleurs — un par domaine métier (Cart, Order, Stock, Partner, User, Dashboard...)
src/Entity/	9 entités Doctrine : User, Partner, Plant, Packaging, Season, Stock, Order, OrderLine, OrderStatusHistory
src/Repository/	10 repositories avec méthodes DQL métier (filtres avancés, alertes, recherche partenaire...)
src/Form/	12 FormType + DTOs de filtrage (OrderFilterData, StockFilterData)

Dossier	Contenu
src/Security/	AppCustomAuthenticator, UserChecker, CheckPasswordChangeSubscriber
src/Service/	CartService (session), OrderNotificationService (email partenaires)
templates/	Templates Twig organisés par domaine, sidebar et composants réutilisables
var/backups/	Exports SQL périodiques de la base de données

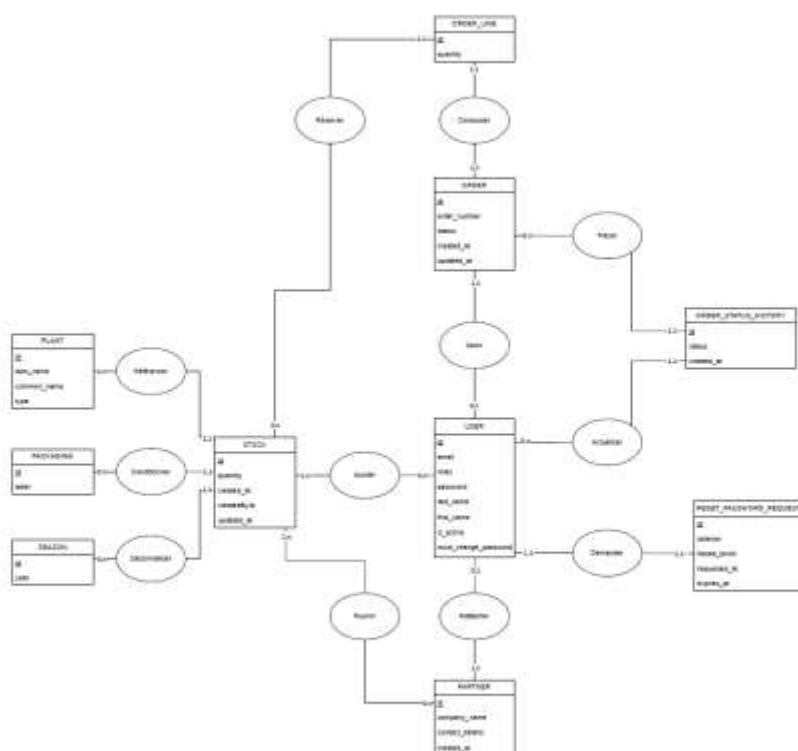
3. Modèle de données

Le modèle repose sur neuf entités. La distinction entre stock réel et stock virtuel s'effectue uniquement via le champ partner de l'entité Stock : si partner est NULL, le stock appartient à Pépi+ (réel) ; s'il est renseigné, il appartient à un partenaire fournisseur (virtuel). Cette convention évite de dupliquer la logique de gestion des stocks et permet une vue globale unifiée.

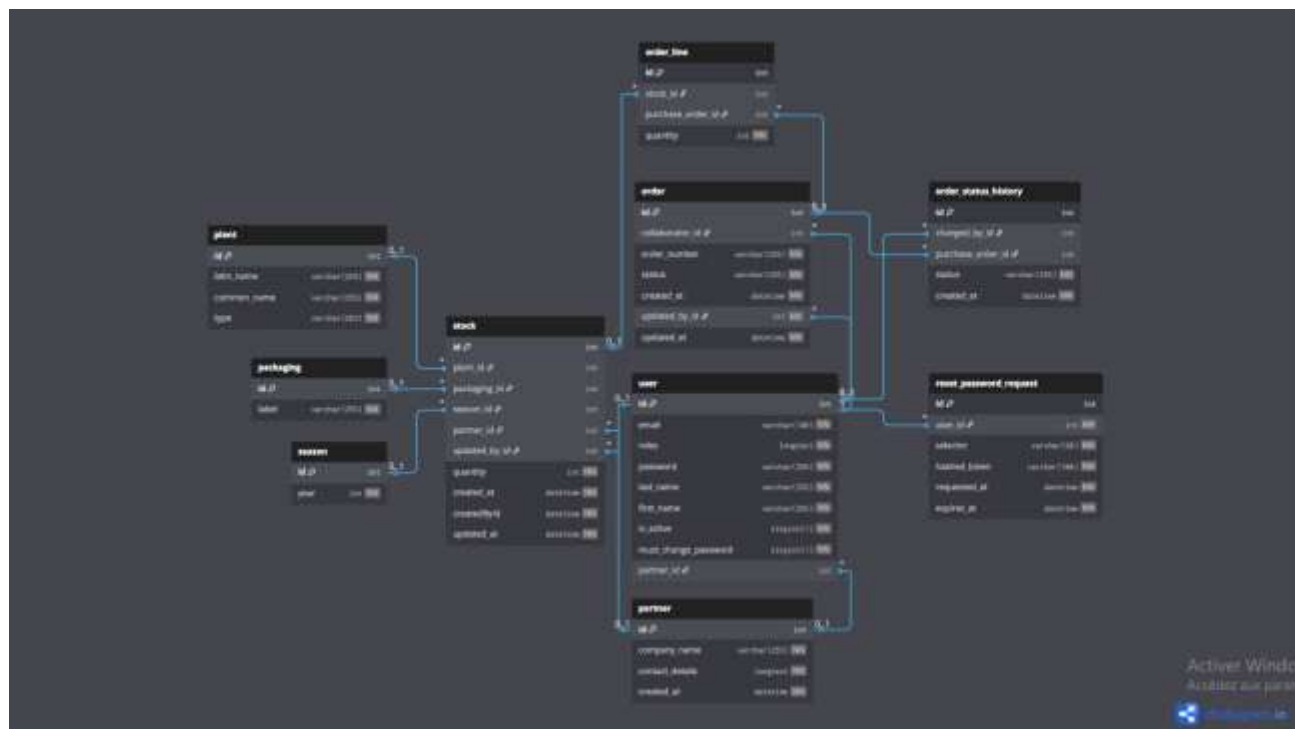
Entité	Rôle	Relations clés
User	Compte utilisateur (ADMIN / COLLABORATOR / PARTNER)	ManyToOne → Partner (nullable)
Partner	Partenaire fournisseur (société)	OneToMany → Stock, OneToMany → User
Plant	Référentiel végétal (nom latin + nom commun)	OneToMany → Stock
Packaging	Conditionnement (ex : pot 2L, motte 20 cm)	OneToMany → Stock
Season	Saison de production (ex : 2024/2025)	OneToMany → Stock
Stock	Unité de stock — partner NULL = réel, NOT NULL = virtuel	ManyToOne → Plant, Packaging, Season, Partner
Order	Commande client (numéro CMD-YYYY-NNN, statut, dates)	OneToMany → OrderLine, OrderStatusHistory
OrderLine	Ligne de commande (quantité réservée + stock source)	ManyToOne → Order, ManyToOne → Stock (onDelete: SET NULL)
OrderStatusHistory	Audit trail des changements de statut	ManyToOne → Order, ManyToOne → User

Le choix onDelete: SET NULL sur la relation OrderLine → Stock est délibéré : si un stock est supprimé (plant retiré du catalogue, partenaire hors contrat), les lignes de commandes historiques sont conservées avec un stock à NULL plutôt que d'être détruites. Cela répond à l'obligation réglementaire de traçabilité sur les commandes passées.

MVC :



MLD :



4. État d'avancement

L'ensemble des exigences fonctionnelles MUST définies dans la Spécification Fonctionnelle v1.3 ont été implémentées et testées. Les deux exigences SHOULD ont également été réalisées.

Priorité	Module	Statut
MUST	Authentification, rôles, reset, mustChangePassword (EF-AUTH)	✓ Réalisé
MUST	Tableau de bord collaborateur et partenaire (EF-DASH)	✓ Réalisé
MUST	Stock global, filtres, traçabilité, gestion interne (EF-STOCK)	✓ Réalisé
MUST	Panier, réservation, modification, annulation, livraison, historique (EF-CMD)	✓ Réalisé
MUST	Référentiel plants, conditionnements, saisons (EF-REF)	✓ Réalisé
MUST	Gestion utilisateurs, partenaires, profil (EF-ADMIN)	✓ Réalisé
MUST	Espace partenaire stock + réservations (EF-PART)	✓ Réalisé
SHOULD	Ajout de ligne directe sur commande existante (EF-CMD-05)	✓ Réalisé
SHOULD	Page profil utilisateur (EF-ADMIN-03)	✓ Réalisé

5. Fonctionnalités réalisées

5.1 Authentification et sécurité des accès

L'authentification repose sur un authenticator personnalisé (AppCustomAuthenticator) qui gère la connexion par email/mot de passe, la redirection post-connexion selon le rôle, et la conservation du dernier email saisi en cas d'erreur. Trois rôles distincts sont définis dans security.yaml via access_control : ADMIN, COLLABORATOR et PARTNER, avec des périmètres d'accès strictement séparés.

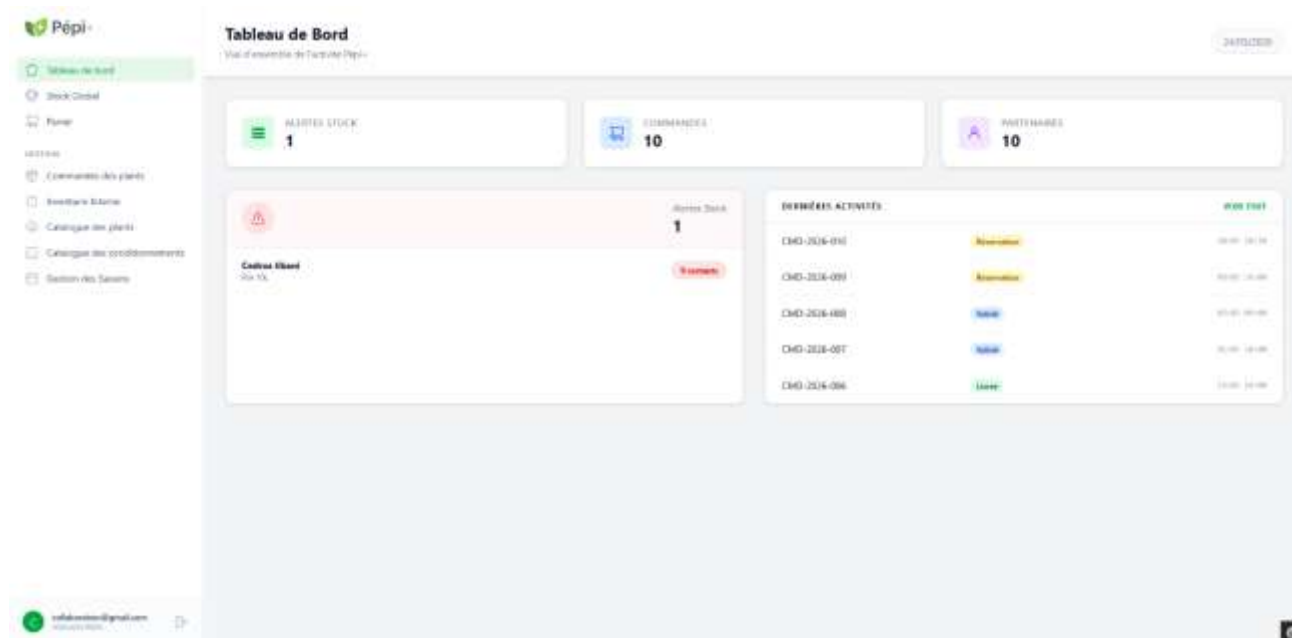
- UserChecker : vérifie que le compte est actif (isActive = true) avant même la validation du mot de passe — en cas de compte désactivé, une exception explicite est levée sans exposer si c'est le mot de passe ou le compte qui pose problème.
- CheckPasswordChangeSubscriber : s'abonne à KernelEvents::REQUEST et intercepte chaque requête principale. Si le flag mustChangePassword est actif (positionné par l'admin lors d'une réinitialisation), l'utilisateur est redirigé vers le formulaire de changement de mot de passe quelle que soit la page demandée.
- Réinitialisation de mot de passe : token aléatoire à usage unique, expiré après 24h, géré par symfonycasts/reset-password-bundle. Le token est stocké haché en base, pas en clair.
- Session expirée après 30 minutes d'inactivité. Protection CSRF active sur tous les formulaires Symfony via token automatique.



5.2 Tableau de bord

Le tableau de bord est la première page affichée après connexion. Son contenu est conditionnel selon le rôle détecté via `isGranted('ROLE_PARTNER')` dans `DashboardController`.

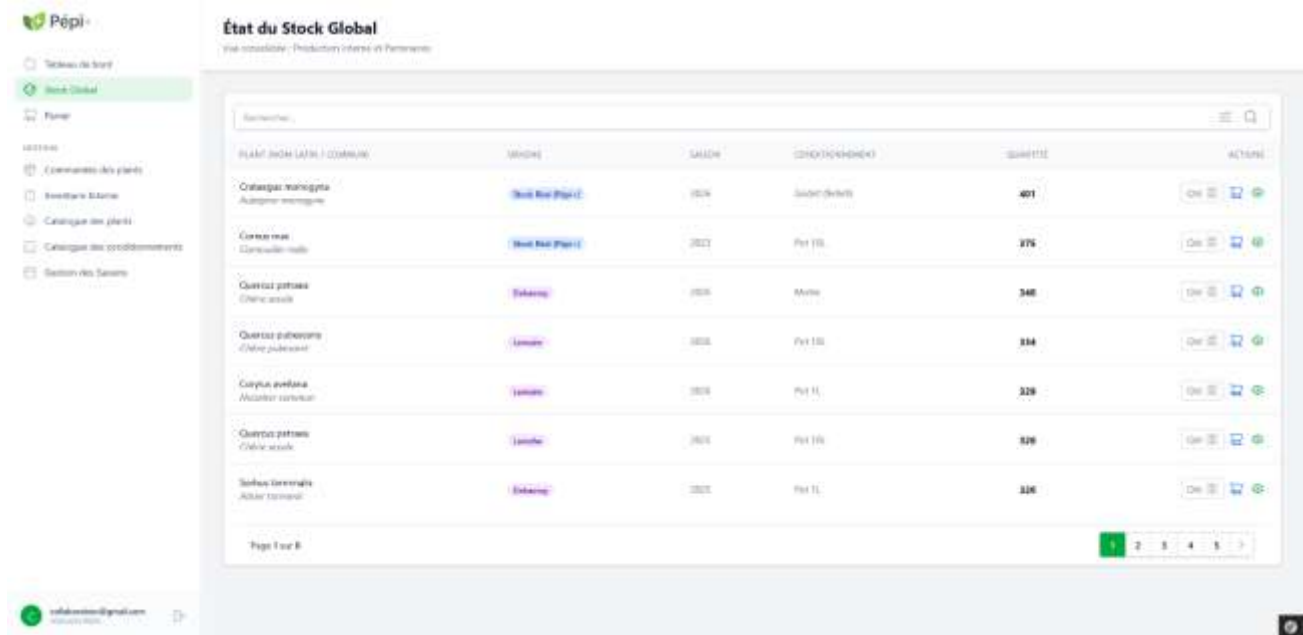
- Vue Admin / Collaborateur : compteurs globaux (total commandes, total partenaires), liste des stocks critiques (quantité < 10 unités, seuil défini dans `StockRepository::findLowStockAlert()`), et les 5 dernières commandes triées par date de création décroissante.
- Vue Partenaire : restreinte à ses propres données — ses stocks, ses stocks critiques via `findLowStockAlertByPartner()`, et ses 5 commandes récentes via `findRecentOrdersByPartner()`.



5.3 Gestion des stocks

Deux vues de stock coexistent selon le profil :

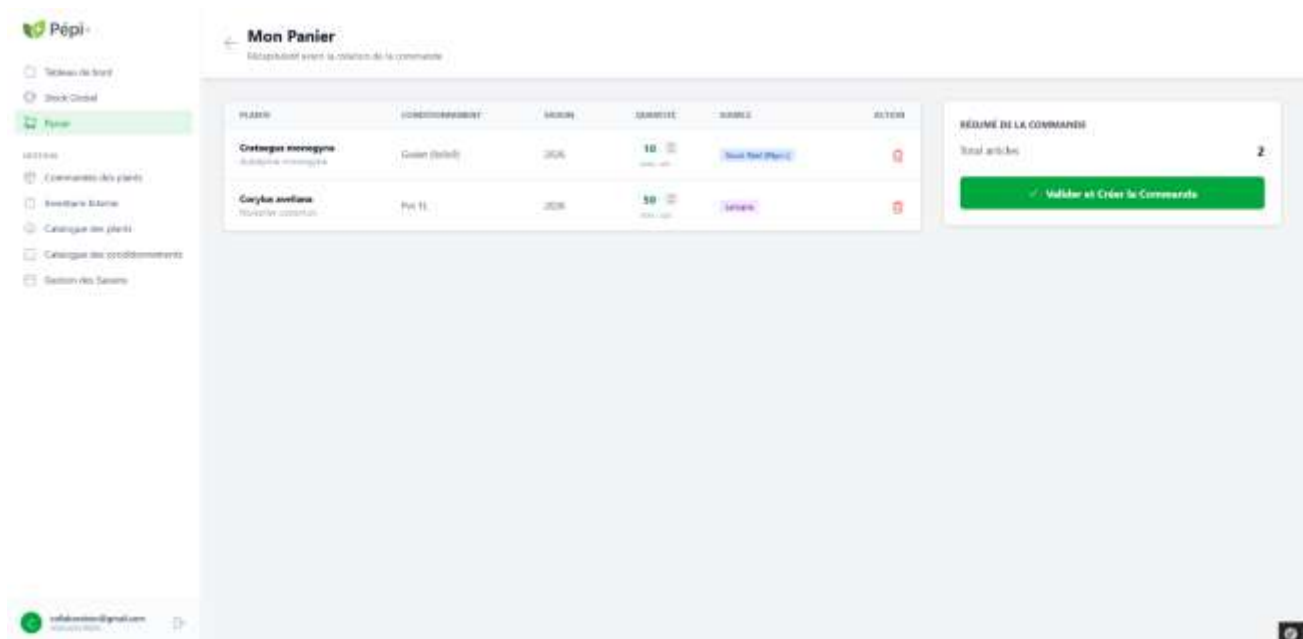
- Vue globale `/stock/global` : affiche tous les stocks (réels + virtuels) dans un tableau paginé à 7 entrées par page. Chaque ligne indique le plant (nom latin + commun), l'origine (badge bleu « Stock Réel Pépi+ » ou badge violet avec le nom du partenaire), la saison, le conditionnement et la quantité disponible. Un formulaire de filtres avancés (`StockFilterType`) permet la recherche textuelle (nom latin, nom commun, partenaire, saison) et le filtrage par plage de quantité. Les quantités inférieures à 10 sont affichées en rouge gras.
- Vue interne `/stock/gestion` : uniquement les stocks Pépi+ (partner IS NULL). Lors de la création, si un stock identique (même plant, même conditionnement, même saison) existe déjà, les quantités sont cumulées automatiquement plutôt que de créer un doublon.



5.4 Panier et création de commandes

Le processus de commande se déroule en deux étapes : constitution du panier puis validation.

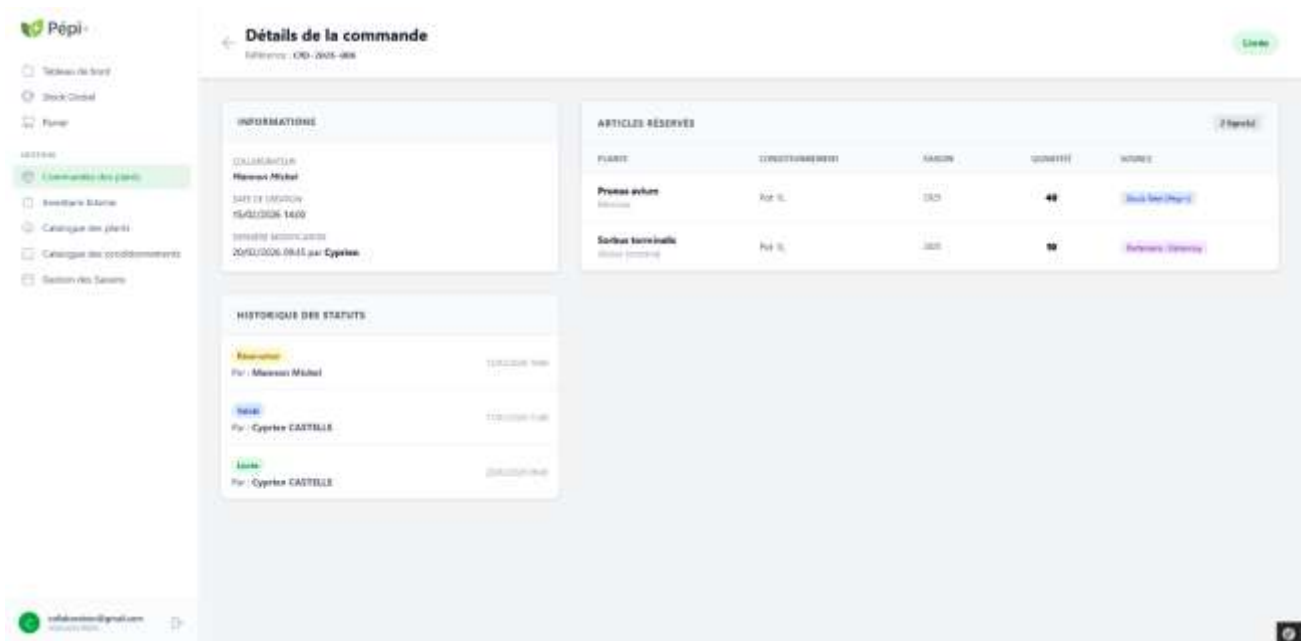
- Constitution : depuis la vue stock global, le collaborateur saisit une quantité et clique sur « Ajouter au panier ». Le CartService stocke le panier en session PHP sous la forme d'un tableau associatif [stockId => quantity]. Plusieurs ajouts sont possibles, les quantités s'additionnent si le même stock est ajouté plusieurs fois.
- Panier /cart : le collaborateur visualise les lignes, peut modifier les quantités (automatiquement plafonnées au stock disponible en base au moment de la modification) ou supprimer des lignes.
- Validation /cart/validate : crée l'entité Order avec le statut Réservation et un numéro au format CMD-YYYY-NNN (généré séquentiellement), crée les OrderLine correspondantes, décrémente immédiatement chaque stock source, enregistre le premier OrderStatusHistory. Si entre l'ajout au panier et la validation la quantité disponible a diminué, elle est ajustée automatiquement avec un message d'avertissement.



5.5 Cycle de vie des commandes

Flux de statuts autorisé : **Réservation** → **Validé** → **Livrée** ou **Annulée**. Une commande Livrée ne peut plus être modifiée.

- Modification du statut : formulaire sur `/order/edit/{id}` — la transition est validée côté contrôleur. Il est également possible d'ajouter directement une ligne à une commande en Réservation via `/order/{id}/add-line-direct` avec une recherche de stock par texte.
- Livraison `/order/{id}/deliver` : passage au statut Livrée uniquement. Aucune modification de stock — celui-ci a déjà été décrémenté à la réservation. La livraison physique est un acte hors système.
- Annulation `/order/{id}/cancel` : impossible si statut Livrée. Pour chaque OrderLine, la quantité réservée est restituée au stock source (réel ou virtuel). Le statut passe à Annulée et un OrderStatusHistory est créé.
- Historique complet : chaque changement de statut enregistre un OrderStatusHistory avec la date et l'utilisateur responsable. Consultable depuis `/order/show/{id}`.



5.6 Espace partenaire et administration

- Partenaire — Mon stock `/partner/my-stock/liste` : CRUD complet du stock virtuel par plant, conditionnement et saison. Paginé à 8 entrées par page. Le champ partner est automatiquement renseigné avec l'entité Partner liée au compte connecté — un partenaire ne peut gérer que son propre stock.
- Partenaire — Mes réservations `/partner/my-reservations/liste` : liste des commandes portant sur ses stocks, dédoublées via `OrderRepository::searchOrdersByPartner()`. Chaque ligne représente une commande unique avec les plants réservés affichés en badges inline (ex : Quercus petraea ×10) et le statut de la commande. Vue en lecture seule.
- Admin — Utilisateurs `/user` : CRUD complet avec activation/désactivation de compte, attribution du rôle (ADMIN, COLLABORATOR ou PARTNER), association à un partenaire si PARTNER, réinitialisation du mot de passe (provisoire Password123! + flag `mustChangePassword` activé).
- Admin — Référentiel : CRUD des plants (nom latin + commun), conditionnements, saisons et partenaires. Chaque entité est paginée et dispose d'un formulaire de création/édition dédié.

6. Identifiants et mise en route

6.1 Comptes de test

Les comptes suivants sont créés par les fixtures Doctrine (doctrine:fixtures:load). Ils permettent de tester chaque espace de l'application sans configuration supplémentaire.

Rôle	Email	Mot de passe
ADMIN	castellscyprien@gmail.com	Qye!9IE8\$^EuQhKB
COLLABORATOR	collaborateur@gmail.com	password
PARTNER	partner@gmail.com	password

Chaîne de connexion BDD à renseigner dans .env.local :

```
DATABASE_URL="mysql://pepi_user:Password123@127.0.0.1:3306/pepi?charset=utf8mb4"
```

Chaîne de connexion BDD à renseigner dans .env.local pour l'épreuve E6:

```
DATABASE_URL="mysql://pepi_user:Password123@127.0.0.1:3306/pepi?charset=utf8mb4"
```

URL locale : http://localhost:8000

6.2 Procédure d'installation

1. Cloner le dépôt	git clone https://github.com/CASTELLS-Cyprien/PEPI-PLUS.git
2. Dépendances PHP	composer install
3. Dépendances JS	npm install
4. Créer .env.local	Renseigner DATABASE_URL avec les paramètres MySQL locaux
5. Initialiser la BDD	php bin/console app:setup-database (crée l'utilisateur MySQL, la BDD, exécute les migrations et charge les fixtures automatiquement)
Alternative BDD	Importer directement l'export SQL : var/backups/db_export_XXXX-XX-XX.sql
6. Compiler les assets	npm run build (ou npm run watch pour le mode développement)
7. Lancer le serveur	symfony server:start → http://localhost:8000

6.3 Commandes utiles

Vider le cache	php bin/console cache:clear
Supprimer la BDD	php bin/console doctrine:database:drop --force
Générer la doc PHPDoc	php phpDocumentor.phar -d src/ -t var/api-docs
Corriger le style de code	vendor/bin/php-cs-fixer fix src/